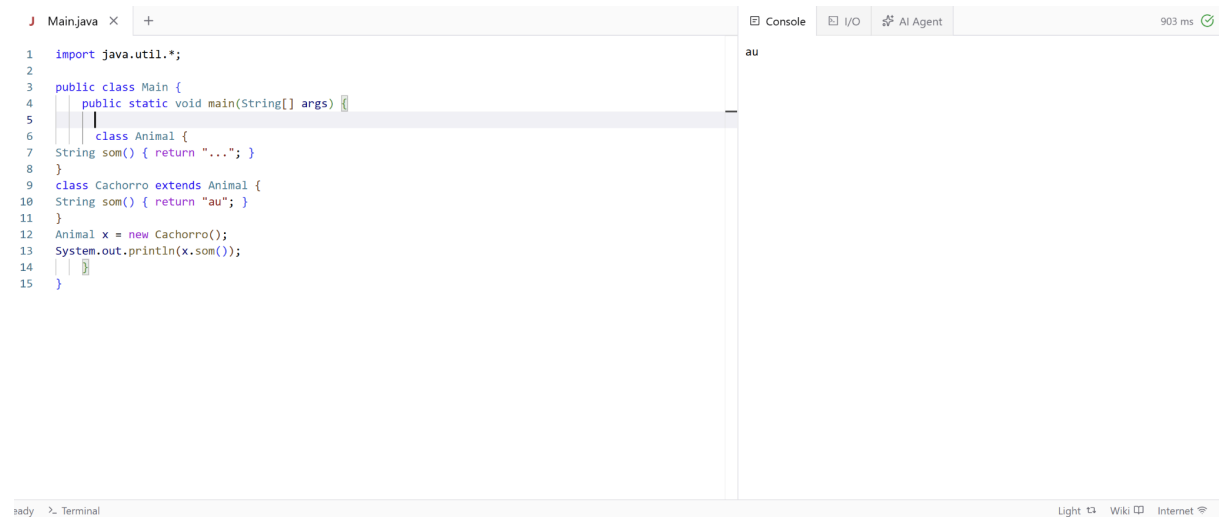


# Atividade Aula 009

## 1 - Java

a -



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Animal' with a 'som()' method that returns "...". A subclass 'Cachorro' extends 'Animal' and overrides 'som()' to return "au". In the 'main' method, an 'Animal' object 'x' is created as a 'Cachorro' instance, and 'x.som()' is called. The console output shows 'au'. The execution time is 903 ms.

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         class Animal {
6             String som() { return "..."; }
7         }
8         class Cachorro extends Animal {
9             String som() { return "au"; }
10        }
11        Animal x = new Cachorro();
12        System.out.println(x.som());
13    }
14 }
15 }
```

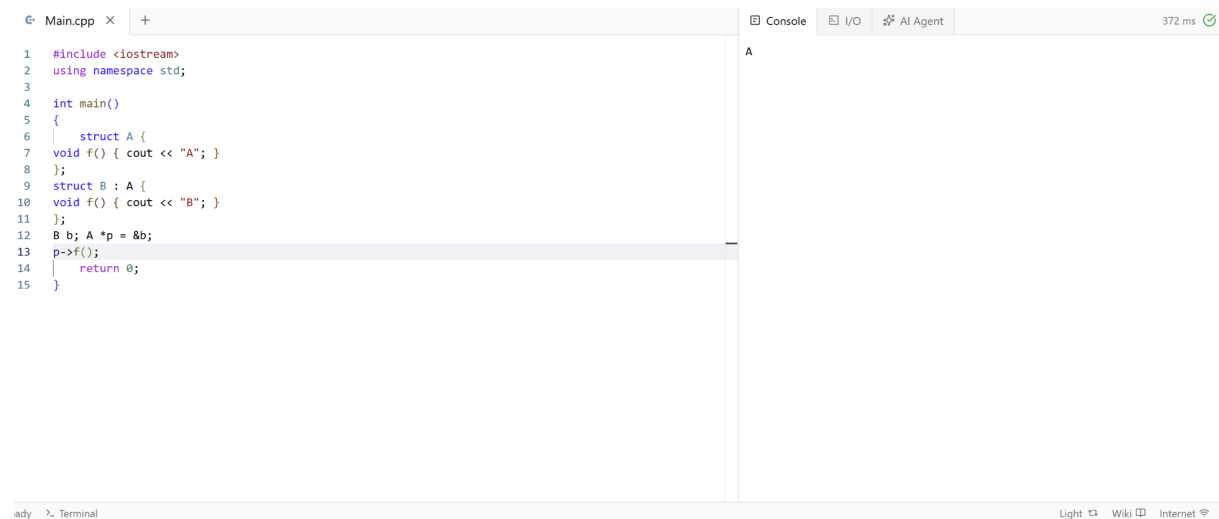
Console: au

903 ms

b - Na execução. Em Java, o despacho dinâmico é o padrão para métodos não estáticos. Portanto, o método executado é resolvido pelo tipo do objeto real em execução (**Cachorro**), mesmo a variável sendo do tipo da superclasse (**Animal**).

## 2 - C++

a -



The screenshot shows a C++ IDE with a file named 'Main.cpp'. The code defines a base struct 'A' with a 'f()' method that prints 'A'. A derived struct 'B' inherits from 'A' and overrides 'f()' to print 'B'. In the 'main' function, a pointer 'p' of type 'A\*' is assigned the address of a 'B' object, and 'p->f()' is called. The console output shows 'A'. The execution time is 372 ms.

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     struct A {
7         void f() { cout << "A"; }
8     };
9     struct B : A {
10         void f() { cout << "B"; }
11     };
12     B b; A *p = &b;
13     p->f();
14     return 0;
15 }
```

Console: A

372 ms

b - Na compilação. Em C++, a vinculação de métodos é estática por padrão. Como a palavra-chave **virtual** não foi utilizada na estrutura base **A**, a decisão é tomada com base no tipo do ponteiro (**A\***), ignorando que o objeto real era **B**.

### 3 - Java

a -

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         class A { String nome = "A";
7         String getNome() { return nome; } }
8         class B extends A { String nome = "B";
9         String getNome() { return nome; } }
10        A x = new B();
11        System.out.println(x.nome + " " + x.getNome());
12    }
13 }
14 }
```

Console I/O AI Agent 974 ms

A B

b - O comportamento é misto. O acesso direto ao atributo (`x.nome`) é decidido na **compilação** com base no tipo da referência (`A`), o que retorna "A". A chamada do método (`x.getNome()`), no entanto, passa por vinculação **dinâmica** e é decidida na **execução**, chamando o método sobrescrito no objeto instanciado (`B`), o que retorna "B".

### 4 - Python

a -

```
1 class Contador:
2     total = 0
3     def __init__(self):
4         Contador.total += 1
5         self.id = Contador.total
6
7 a = Contador(); b = Contador()
8 print(a.id, b.id, a.total)
```

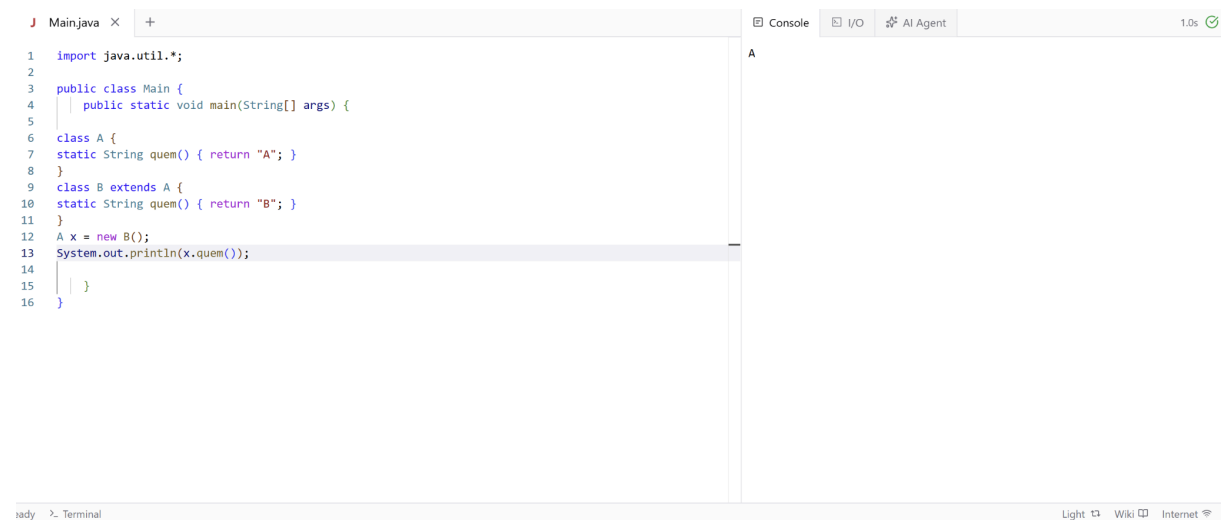
Console I/O AI Agent 131 ms

1 2 2

b - Na execução. Em Python, a vinculação é sempre dinâmica. A variável de classe `Contador.total` é incrementada a cada nova instância criada. Ao final, `a.id` possui valor 1, `b.id` possui valor 2, e `a.total` acessa o estado final e compartilhado da variável na classe, que é 2.

## 5 - Java

a -



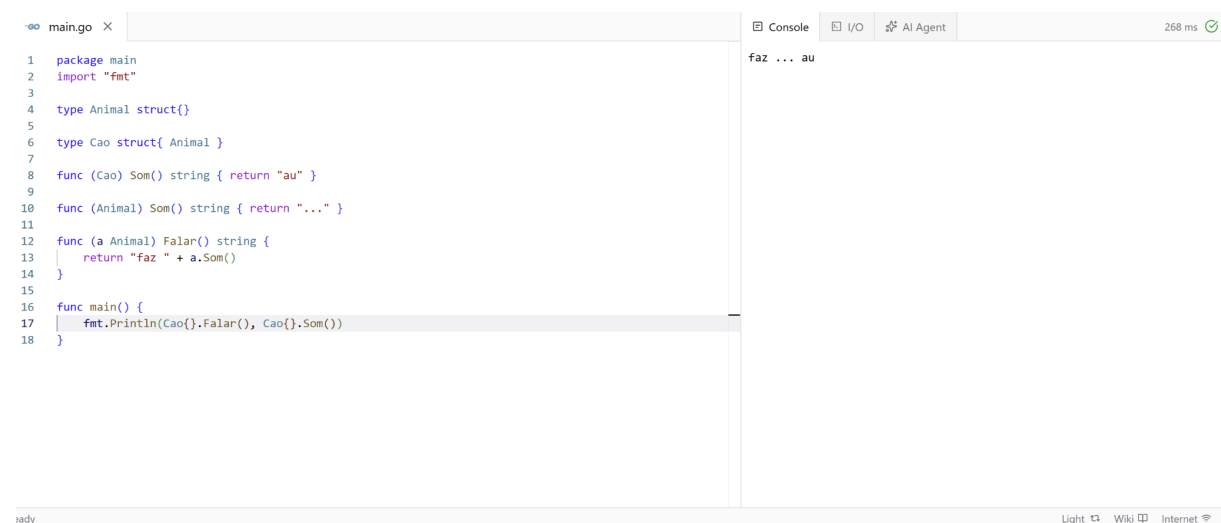
```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         class A {
7             static String quem() { return "A"; }
8         }
9         class B extends A {
10             static String quem() { return "B"; }
11         }
12         A x = new B();
13         System.out.println(x.quem());
14     }
15 }
16 }
```

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a 'Main' class with a 'main' method. Inside 'main', it defines two inner classes: 'A' and 'B'. 'A' has a static method 'quem()' that returns 'A'. 'B' extends 'A' and overrides 'quem()' to return 'B'. In the 'main' method, an instance of 'B' is created and assigned to a variable 'x' of type 'A'. Then, 'x.quem()' is called, which prints 'B' to the console. The IDE interface includes a 'Console' tab on the right, a 'Terminal' tab at the bottom, and a status bar at the bottom right showing 'Light', 'Wiki', and 'Internet' icons.

b - Na compilação. Métodos que utilizam o modificador **static** em Java não participam do mecanismo de vinculação dinâmica. Assim, a escolha de qual método chamar é feita rigidamente pela classe declarada da variável de referência (**A**), independentemente do objeto real.

## 6 - Go

a -



```
1 package main
2 import "fmt"
3
4 type Animal struct{}
5
6 type Cao struct{ Animal }
7
8 func (Cao) Som() string { return "au" }
9
10 func (Animal) Som() string { return "..." }
11
12 func (a Animal) Falar() string {
13     return "faz " + a.Som()
14 }
15
16 func main() {
17     fmt.Println(Cao{}.Falar(), Cao{}.Som())
18 }
```

The screenshot shows a Go IDE with a file named 'main.go'. The code defines two structs: 'Animal' and 'Cao'. 'Cao' is a struct that contains an 'Animal' struct. 'Animal' has a method 'Som()' that returns '...'. 'Cao' has a method 'Som()' that returns 'au'. In the 'main' function, 'Cao{}.Falar()' and 'Cao{}.Som()' are called, which prints 'faz ... au' to the console. The IDE interface includes a 'Console' tab on the right, a 'Terminal' tab at the bottom, and a status bar at the bottom right showing 'Light', 'Wiki', and 'Internet' icons.

b - Na compilação. A linguagem Go utiliza composição em vez de herança de classes e não faz despacho dinâmico para métodos em receivers de structs nativas da mesma maneira. O método **Falar()** é promovido da struct aninhada **Animal** para **Cao**, mas ao ser executado dentro de **Animal**, ele aciona rigidamente o **Som()** de **Animal** imprimindo "faz ...". A chamada explícita **Cao{}.Som()** aciona o método diretamente atrelado a **Cao**, imprimindo "au".